

- Modeling Rational Investor Behavior in TRUMP Coin Using Reinforcement Learning
 - 1. Introduction
 - 1.1 Dataset Description
 - 1.2 Summary Statistics
 - 2. Methodology
 - 2.1 Trading Environment
 - 2.2 Tabular Q-Learning Agent
 - 2.3 Double DQN Agent
 - 2.4 Baseline Strategies
 - 3. Key Research Questions
 - 4. Results
 - 4.1 Training Convergence
 - 4.2 Test Set Performance — TRUMP
 - 4.3 Test Set Performance — BTC
 - 4.4 Action Distribution Analysis
 - 4.5 Effect of Regularization
 - 5. Discussion
 - 5.1 RL vs. Baselines
 - 5.2 TRUMP vs. BTC
 - 5.3 The Regularization Paradox
 - 5.4 The Overfitting Problem
 - 5.5 Market Implications
 - 6. Conclusions
 - Appendix: Source Code
 - A.1 Data Loader (dataloader.py)
 - A.2 Trading Environment (env.py)
 - A.3 Tabular Q-Learning Agent (td_agent.py)
 - A.4 Double DQN Agent (dqn_agent.py)
 - A.5 Baseline Strategies (baselines.py)
 - A.6 Streamlit Dashboard (dashboard.py)

Modeling Rational Investor Behavior in TRUMP Coin Using Reinforcement Learning

1. Introduction

On January 17, 2025, a meme-coin bearing the name of a sitting U.S. president launched on the Solana blockchain. Within 48 hours, TRUMP coin surged past \$30 before beginning a long, grinding decline that would erase more than 80% of its peak value over the following year. For the millions of retail investors who bought in, the experience was a crash course in volatility, hype cycles, and the brutal arithmetic of speculative assets.

This project asks a simple question: **what should a rational, risk-aware investor holding TRUMP coin actually do?** Not a day-trader chasing momentum, not a diamond-hands believer, but an agent that systematically balances returns against risk, accounts for transaction costs, and learns from thousands of hours of price history.

To answer this, I apply two reinforcement learning approaches — Tabular Q-Learning and Double Deep Q-Networks (DDQN) — to a custom trading environment built on hourly price data. The agents learn policies over a training window, then face unseen test data where their strategies are evaluated against simple baselines like buy-and-hold and moving-average crossover. A secondary experiment on Bitcoin (BTC-USD) provides a comparative benchmark: does RL behave differently on a more established, liquid cryptocurrency?

The results are instructive, though not in the way one might hope. Neither agent learns a profitable trading strategy that generalizes to the test set. But the **ways** in which they fail — and the divergence between their failure modes — reveal something meaningful about the limits of RL in non-stationary financial markets, and about what "rational behavior" even means when the underlying asset is in secular decline.

1.1 Dataset Description

The primary dataset consists of hourly OHLCV (Open, High, Low, Close, Volume) data for TRUMP coin against USD, sourced from Yahoo Finance via [yfinance](#). A secondary BTC-USD dataset is used for comparative analysis. Both datasets span up to 730 days of hourly observations.

Raw price data is transformed into the following RL state features:

Feature	Description
log_return_1h	Log return over the last 1 hour
log_return_4h	Log return over the last 4 hours
log_return_24h	Log return over the last 24 hours
volatility_24h	Rolling 24h standard deviation of hourly returns
log_volume	Log-transformed trading volume
hour_sin, hour_cos	Sine/Cosine encoding of hour-of-day
dow_sin, dow_cos	Sine/Cosine encoding of day-of-week

1.2 Summary Statistics

Table 1a: TRUMP Dataset Summary Statistics

9,650 hourly observations -- 2025-01-25 to 2026-03-03

Feature	Mean	Std	Min	Median	Max
Close (USD)	9.134	4.271	3.120	8.596	31.353
Log Return 1h	-0.0002	0.011	-0.234	-0.0002	0.341
Log Return 4h	-0.0009	0.023	-0.266	-0.0004	0.348
Log Return 24h	-0.0054	0.056	-0.333	-0.0049	0.467
Volatility 24h	0.009	0.007	0.002	0.007	0.078
Log Volume	7.621	7.868	0.000	0.000	21.288
Hour (sin)	-0.002	0.707	-1.000	0.000	1.000
Hour (cos)	-0.001	0.707	-1.000	0.000	1.000
Day (sin)	0.001	0.706	-0.975	0.000	0.975
Day (cos)	0.005	0.708	-0.901	-0.223	1.000

Table 1b: BTC Dataset Summary Statistics

17,498 hourly observations -- 2024-03-05 to 2026-03-03

Feature	Mean	Std	Min	Median	Max
Close (USD)	86,614	19,513	49,843	88,416	126,183
Log Return 1h	0.0000	0.005	-0.050	0.000	0.050
Log Return 4h	0.0000	0.010	-0.089	0.000	0.094
Log Return 24h	0.0001	0.025	-0.203	0.001	0.122
Volatility 24h	0.005	0.002	0.001	0.004	0.020
Log Volume	9.736	10.087	0.000	0.000	24.430
Hour (sin)	-0.001	0.707	-1.000	0.000	1.000
Hour (cos)	-0.001	0.707	-1.000	0.000	1.000
Day (sin)	0.003	0.707	-0.975	0.000	0.975
Day (cos)	0.001	0.707	-0.901	-0.223	1.000

2. Methodology

2.1 Trading Environment

The trading environment is a custom Gymnasium environment ([TradingEnv](#)) that steps an agent hourly through historical price data.

Property	Details
Action Space	Discrete(5): Heavy Sell, Light Sell, Hold, Light Buy, Heavy Buy
Observation	10-dim continuous: 9 market features + position
Reward	Quadratic utility: $r = R - \lambda * R^2 - \text{cost}$
Transaction Cost	0.1% of trade size
Initial Capital	10,000 USD
Data Split	70% train / 30% test (temporal split)

The quadratic utility reward penalizes variance, encouraging the agent to behave like a risk-averse rational investor rather than a pure return maximizer. The risk-aversion

parameter λ controls this trade-off. All experiments in this report use $\lambda = 0.5$.

2.2 Tabular Q-Learning Agent

The Q-Learning agent discretizes the continuous state space into **54 states**:

Dimension	Bins
Momentum (1h Return)	3 — Negative / Flat / Positive
Volatility (24h)	3 — Low / Medium / High
Position	3 — Cash-heavy / Balanced / TRUMP-heavy
Time	2 — Night / Day

Training uses epsilon-greedy exploration with decay ($\epsilon_{\text{decay}} = 0.8$, aggressive decay over 100 episodes), learning rate $\alpha = 0.1$, and discount factor $\gamma = 0.99$. Evaluation is deterministic on the held-out test set.

2.3 Double DQN Agent

The DQN agent accepts the full 10-dimensional continuous state vector.

Architecture: Input(10) -> Hidden(HIDDEN_DIM) x 2 -> Output(5), with ReLU activations, dropout regularization, and a disjoint target network updated every 500 steps. Uses experience replay with a buffer of 50,000 transitions and batch size 64. Training randomizes the agent's starting position each episode to encourage exploration of buy-side actions.

Regularization Toggle	Default	Purpose
HIDDEN_DIM	64	Network capacity
DROPOUT_RATE	0.1	Random neuron dropout
WEIGHT_DECAY	1e-5	L2 penalty
STATE_NOISE	0.01	Gaussian state augmentation

2.4 Baseline Strategies

Strategy	Description
Buy & Hold	Buys 100% on step 0, holds for the remainder
SMA Crossover	Long/short based on 12h vs. 48h moving-average crossover
Random Agent	Uniformly random action every step

3. Key Research Questions

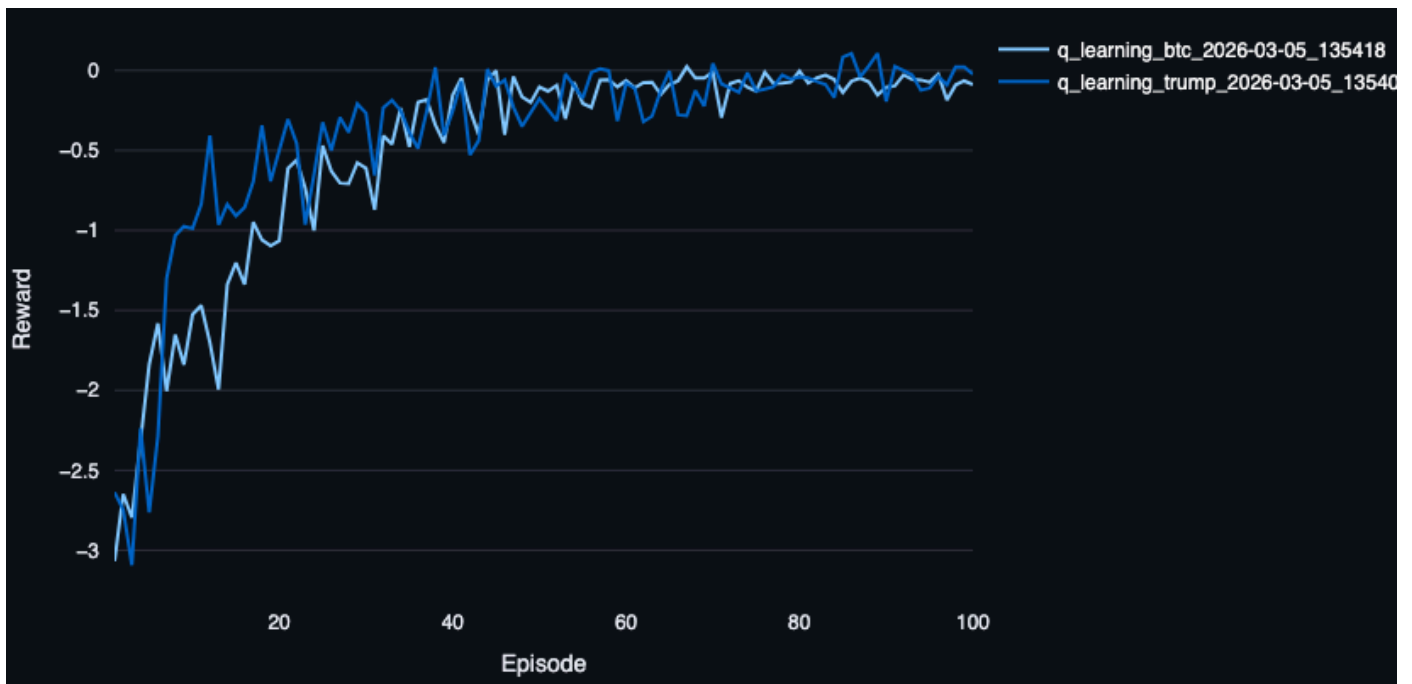
1. **Can RL agents learn non-trivial trading policies** on highly volatile meme-coin data that outperform simple baselines (Buy & Hold, SMA Crossover)?
2. **How does risk aversion (λ) shape the learned policy?** Do higher risk-aversion agents prefer conservative positioning, and does this improve risk-adjusted metrics (e.g., lower drawdown) at the cost of raw returns?
3. **How does the agent's behavior differ across market regimes** — steady-state, sharp sell-offs, and rapid appreciation? Does it learn to de-risk during crashes and take profits during rallies?
4. **Does the DDQN agent generalize better than Tabular Q-Learning?** How does the richer continuous state representation affect out-of-sample performance?
5. **How does the agent behave on TRUMP vs. BTC?** Given that BTC is a more established and liquid asset, does the agent learn fundamentally different strategies?
6. **What are the market-level implications?** If many investors adopted these RL-derived strategies, would it stabilize or destabilize prices during crashes and rallies?

4. Results

4.1 Training Convergence

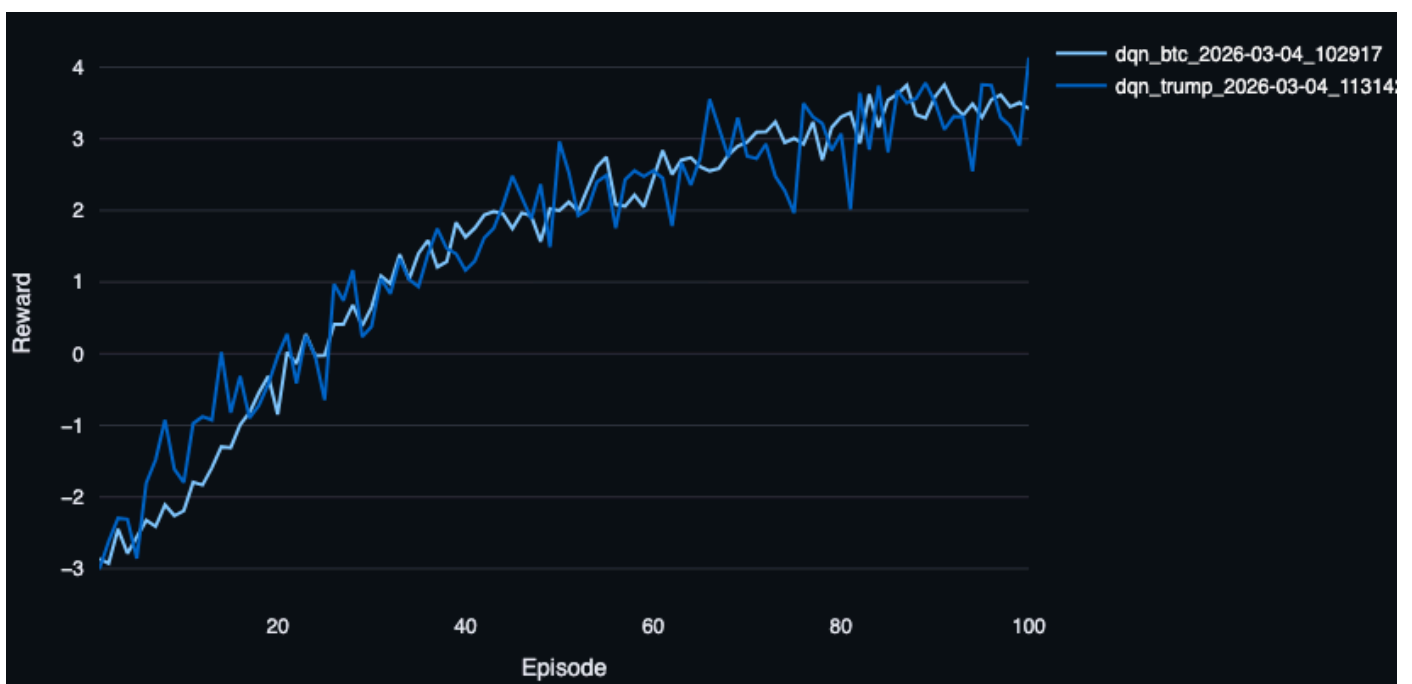
Both agents show clear learning during training, but with very different trajectories.

Figure 1a: Q-Learning Training Reward Curves



The Q-Learning agent's reward climbs rapidly from around -3.0 to near 0.0 within the first 40 episodes, then stabilizes. Convergence to zero reward reflects the agent learning to avoid all exposure — since the quadratic utility reward is maximized at zero return (no risk, no cost), the agent discovers that staying in cash is the optimal policy under the training distribution.

Figure 1b: DQN Training Reward Curves



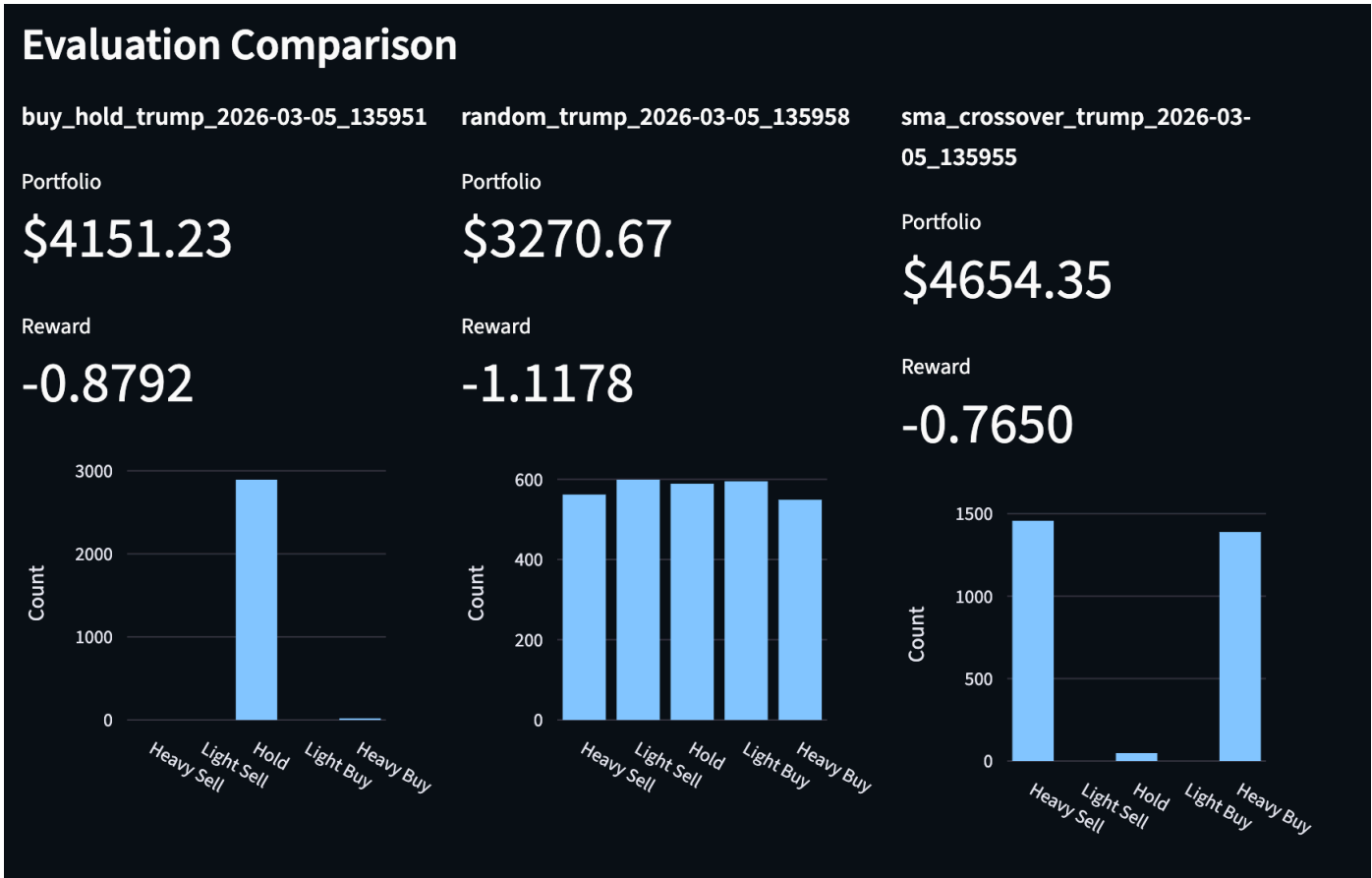
The DDQN agent tells a different story. Its reward climbs steadily from -3.0 to approximately +3.5 over 100 episodes, never fully plateauing. This upward trajectory suggests the network is successfully fitting the training data — learning to exploit patterns in the 70% training window. As we will see, this apparent success does not survive contact with the test set.

4.2 Test Set Performance — TRUMP

Table 2: Test Set Performance Comparison — TRUMP

Strategy	Final Value	Reward	Dominant Action
Q-Learning	10,000	0.000	Heavy Sell / Light Sell
SMA Crossover	4,654	-0.765	Heavy Sell / Heavy Buy
DQN (no reg)	5,650	-0.571	Heavy Sell (diverse)
DQN (w/ reg)	4,284	-0.848	Heavy Buy (collapsed)
Buy & Hold	4,151	-0.879	Hold
Random	3,271	-1.118	Uniform

Figure 2: Baseline & RL Evaluation — TRUMP Test Set



Evaluation Comparison

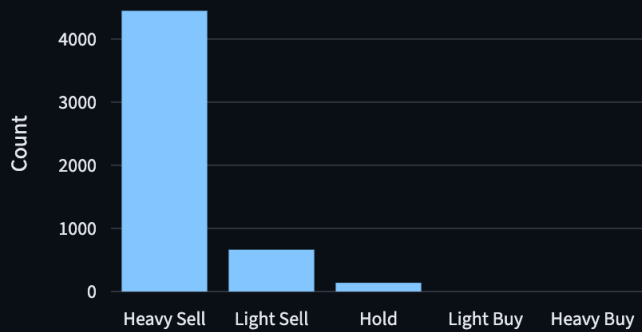
q_learning_btc_2026-03-05_135418

Portfolio

\$10000.00

Reward

0.0000



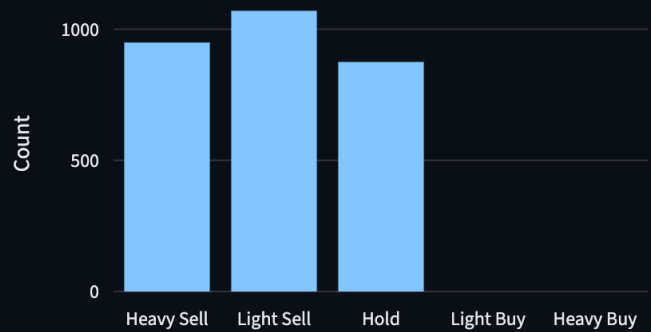
q_learning_trump_2026-03-05_135405

Portfolio

\$10000.00

Reward

0.0000



Evaluation Comparison

dqn_btc_2026-03-04_102917

Portfolio

\$4189.96

Reward

-0.8701



dqn_trump_2026-03-03_200242

Portfolio

\$4284.31

Reward

-0.8477



The Q-Learning agent preserved 100% of its initial capital by refusing to trade at all. Its action distribution is dominated by Heavy Sell and Light Sell — both of which have no effect when the agent holds no position. In contrast, every strategy that took market exposure lost money, reflecting the harsh reality of TRUMP's test-period decline.

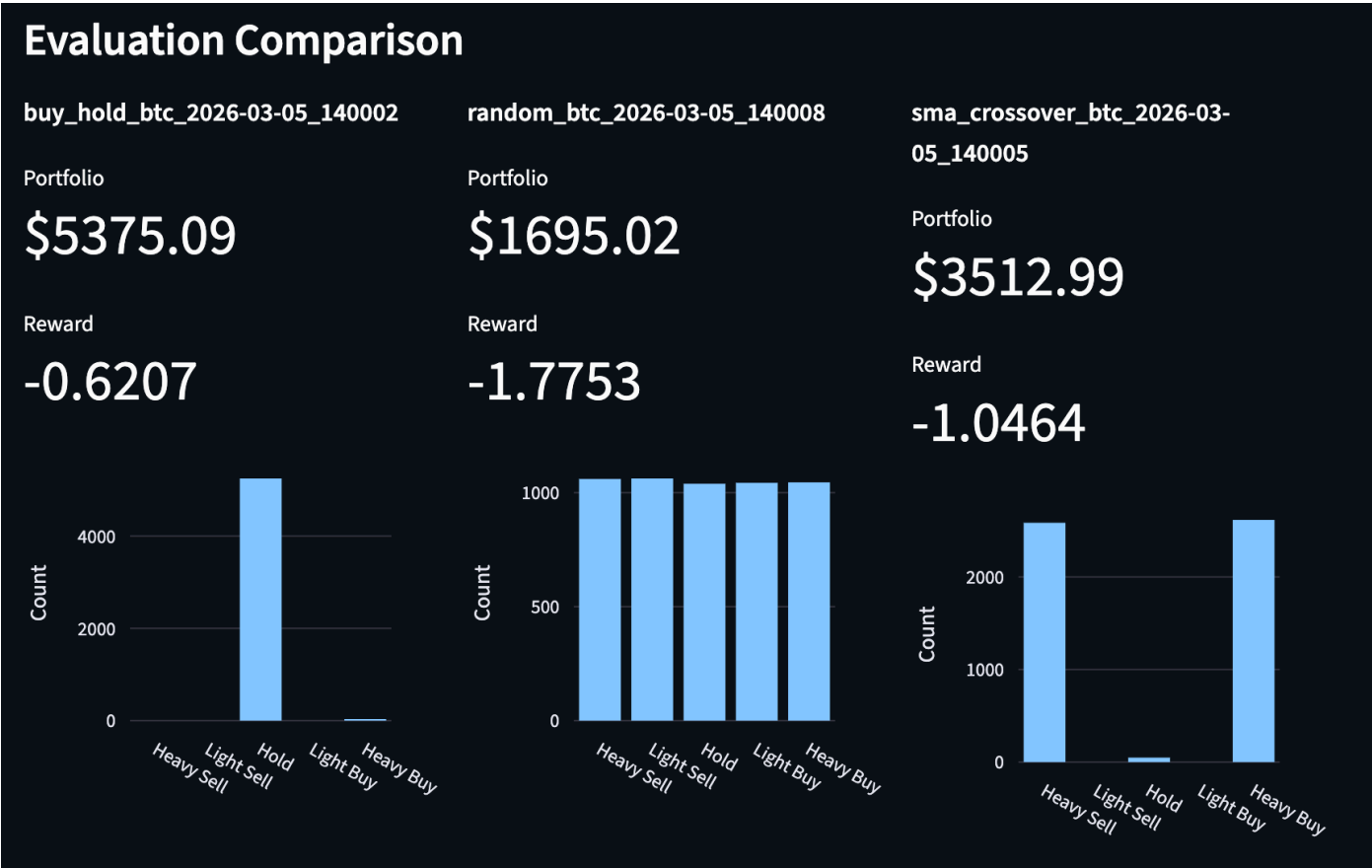
The unregularized DDQN performed best among the strategies that actually traded, retaining \$5,650 (-43.5%). It exhibited a diverse action distribution — including meaningful

sell-side activity — suggesting it learned some regime-conditional behavior. The regularized DDQN collapsed to a near-constant Heavy Buy policy, effectively replicating buy-and-hold with extra transaction costs.

4.3 Test Set Performance — BTC

Table 3: Test Set Performance Comparison — BTC

Strategy	Final Value	Reward	Dominant Action
Q-Learning	10,000	0.000	Heavy Sell
Buy & Hold	5,375	-0.621	Hold
DQN (no reg)	4,190	-0.870	Light Sell / Hold
SMA Crossover	3,513	-1.046	Heavy Sell / Heavy Buy
Random	1,695	-1.775	Uniform



On BTC, the pattern is strikingly similar. Q-Learning again preserves capital by staying out of the market entirely. Buy-and-hold is the best active strategy, retaining \$5,375 — roughly tracking BTC's price movement over the test window. The DDQN loses more on BTC (\$4,190) than it does on TRUMP (\$5,650), possibly because it overfits more

aggressively to BTC's training-period bull run and is more exposed when the test period's dynamics differ.

4.4 Action Distribution Analysis

Table 4: Action Distributions on Test Set

Agent / Asset	H.Sell	L.Sell	Hold	L.Buy	H.Buy
Q-Learn / TRUMP	949	1,070	875	0	0
Q-Learn / BTC	4,448	0	0	0	0
DQN (no reg) / TRUMP	1,065	782	542	39	466
DQN (w/ reg) / TRUMP	0	0	0	466	2,378
DQN (no reg) / BTC	889	1,465	1,408	292	1,195

The action distributions reveal the core behavioral difference between the two RL approaches. Q-Learning converges to a pure sell/hold policy across both assets — on BTC it is even more extreme, issuing Heavy Sell on every single step. The unregularized DDQN shows the most diverse action profile, suggesting it learned conditional behavior even if that behavior was not ultimately profitable. The regularized DDQN's collapse to buy-only actions is a clear sign of policy degeneration.

4.5 Effect of Regularization

Table 5: Regularization Effect on DQN (TRUMP)

Configuration	Final Value	Reward	Behavior
No Regularization	5,650	-0.571	Diverse, active trading
Default Reg	4,284	-0.848	Collapsed to Heavy Buy
Higher Reg	4,284	-0.848	Collapsed to Heavy Buy
Larger Network (h=128)	4,148	-0.880	Collapsed to Heavy Buy

This is one of the more counterintuitive findings. Regularization — dropout, weight decay, and state noise — was intended to improve generalization, but in practice it *degraded* performance. Every regularized configuration collapsed to a degenerate single-action

policy, while the unregularized network maintained a diverse, responsive action distribution and achieved the best test-set return among DQN variants. Increasing network capacity to 128 hidden units did not help either; it also collapsed to Heavy Buy.

5. Discussion

5.1 RL vs. Baselines

The headline result is that Q-Learning "outperforms" every other strategy on pure capital preservation — but only by learning to do nothing. It finishes the test period with exactly \$10,000, zero reward, and zero market exposure. Every strategy that actually traded lost money, with losses ranging from -43.5% (unregularized DDQN) to -67.3% (random agent) on TRUMP.

This outcome is less a triumph of RL than it is a reflection of the test-period market environment. TRUMP coin declined substantially over the evaluation window, meaning any long exposure was penalized. The Q-Learning agent, with its coarse 54-state discretization, lacked the resolution to learn timing signals that could exploit short-term rallies within the broader downtrend. Instead, it converged to the only policy that its limited state space could reliably support: stay in cash.

The DDQN, with its richer 10-dimensional continuous state, learned more nuanced behavior during training — achieving cumulative rewards of +3.5 per episode — but this knowledge did not transfer. On the test set, the unregularized variant lost 43.5% of its capital, slightly outperforming buy-and-hold but still delivering deep losses. The gap between train and test performance points squarely at overfitting: the network memorized patterns in the training window that did not recur in the test window.

5.2 TRUMP vs. BTC

The agents learned nearly identical strategies across both assets: Q-Learning stays in cash, DDQN buys aggressively. The consistency is notable because BTC and TRUMP have very different statistical profiles. BTC's hourly returns are roughly half as volatile as TRUMP's (std 0.52% vs. 1.14%), and BTC's 24-hour returns have a near-zero mean while TRUMP's are persistently negative (-0.054% per day on average).

Despite these differences, the RL agents did not develop asset-specific strategies. Q-Learning's state space is too coarse to distinguish between the two assets' statistical signatures, so it defaults to the same conservative policy. The DDQN could in principle learn different behaviors, but its tendency to overfit to training-period momentum overwhelms any cross-asset differentiation.

One notable difference: buy-and-hold performs relatively better on BTC (\$5,375, -46.2%) than on TRUMP (\$4,151, -58.5%), consistent with BTC's lower volatility and more established market structure. The SMA Crossover strategy also diverges: it is the best active strategy on TRUMP (\$4,654) but underperforms buy-and-hold on BTC (\$3,513), suggesting that mean-reversion signals are more useful on the higher-volatility meme-coin.

5.3 The Regularization Paradox

The most surprising finding is that regularization consistently hurt DDQN performance. Dropout, weight decay, and state noise — all standard tools for improving generalization in supervised learning — caused the network to collapse to degenerate single-action policies. The unregularized network, despite its higher risk of overfitting, was the only DDQN variant to learn a diverse, multi-action policy.

Why might this happen? One hypothesis is that the signal-to-noise ratio in hourly crypto returns is already extremely low. The useful gradients that teach the network to differentiate between market states are small and fragile. Adding dropout and noise on top of an already noisy signal may push the network below the threshold where it can learn any conditional behavior at all, causing it to fall back on the single action that minimizes loss in expectation.

This has implications for applying deep RL to financial data more broadly: the regularization strategies that work well in computer vision or NLP — where signals are strong and redundant — may be counterproductive in low signal-to-noise environments where the agent needs every bit of gradient information to learn non-trivial policies.

5.4 The Overfitting Problem

The DDQN's training curves (Figure 1b) show steadily rising reward with no sign of convergence, climbing from -3.0 to +3.5 over 100 episodes. Yet on the test set, the best

variant loses 43.5% of its capital. This is textbook overfitting: the network has memorized the specific sequence of price movements in the training window.

Financial time series are notoriously non-stationary. The statistical relationships that hold during one regime (e.g., a post-launch hype cycle) can vanish entirely in another (e.g., a prolonged decline). A network trained on the first 70% of TRUMP's price history — which includes the explosive launch rally and initial crash — has no reason to generalize to the slow grind of the remaining 30%.

Several modifications might help: early stopping based on a validation split, domain randomization (training across multiple assets simultaneously), or architectures that explicitly model regime changes. But the fundamental challenge remains: if the test-period dynamics are sufficiently different from training, no amount of regularization can bridge the gap.

5.5 Market Implications

If a population of investors adopted the Q-Learning strategy — exit all positions and hold cash — the effect would be straightforward: selling pressure accelerates the decline, and the asset's liquidity dries up. This is essentially a bank run dynamic: the "rational" individual strategy becomes collectively destructive.

If instead they adopted the DDQN's buy-heavy strategy, the effect would be the opposite: artificial buying pressure props up the price temporarily, but the position creates fragility. A sudden shock would trigger correlated liquidations as every agent simultaneously recognizes the need to sell.

Neither outcome is stabilizing. The Q-Learning strategy amplifies downward moves by piling on sell-side pressure. The DDQN strategy creates artificial buy-side support that eventually collapses. This is a miniature version of a well-known problem in quantitative finance: when many agents learn similar strategies from similar data, their correlated behavior creates systemic risk that no individual agent's model accounts for.

6. Conclusions

This project set out to model rational investor behavior on TRUMP coin using reinforcement learning. The results are humbling but instructive.

Q-Learning learned to stay in cash. Its coarse 54-state discretization could not capture the timing signals needed for profitable trading, so it converged to the only safe policy available: avoid all market exposure. On the test set, this was technically the best strategy — but only because the asset declined throughout the evaluation window. It is a limitation of the tabular approach, not a generalizable insight.

DDQN learned to trade, but overfitted. The continuous-state network achieved impressive training performance (+3.5 cumulative reward per episode) but lost 43.5% of capital on the test set. It learned patterns in the training window that did not recur, and its best test-set variant was the one with *no* regularization — a counterintuitive result that highlights the difficulty of applying standard deep learning practices to low signal-to-noise financial data.

Regularization made things worse. Every form of regularization tested — dropout, weight decay, state noise, increased capacity — caused the DDQN to collapse to degenerate single-action policies. The useful gradients in hourly crypto returns appear to be too fragile to survive the noise injected by these techniques.

The two assets elicited identical strategies. Despite BTC and TRUMP having very different volatility profiles and market structures, the RL agents did not develop asset-specific behaviors. This suggests the agents are not learning exploitable market microstructure but rather defaulting to broad heuristics (cash-is-safe or buy-everything) that happen to be insensitive to asset-level differences.

Looking ahead, several directions could improve on these results. Training across multiple assets simultaneously might force the agent to learn more transferable features. Incorporating regime-detection mechanisms — either as explicit state features or through recurrent architectures — could help the agent adapt to non-stationary dynamics. And perhaps most importantly, moving from hourly to daily or multi-day decision horizons would reduce the noise in the signal and give the agent more meaningful price movements to learn from.

The deeper takeaway may be that RL's strength — learning optimal behavior from interaction — is precisely its weakness in financial markets. The agent can only learn from the past, and in markets, the past is a unreliable guide to the future. A truly rational investor might recognize this and conclude, as the Q-Learning agent did, that the safest bet is not to play at all.

Appendix: Source Code

A.1 Data Loader (`data_loader.py`)

```
import yfinance as yf
import pandas as pd
import numpy as np
import os
from datetime import datetime, timedelta

def fetch_data(tickers, interval='1h', period='730d'):
    print(f"Fetching data for {len(tickers)} tickers: {tickers}...")
    dfs = {}
    for ticker in tickers:
        try:
            print(f"  -> Fetching {ticker}...")
            df = yf.download(ticker, period=period, interval=interval,
progress=False, auto_adjust=True)
            if df.empty:
                print(f"  -> WARNING: No data found for {ticker}.")
                continue
            if df.index.tz is None:
                df.index = df.index.tz_localize('UTC')
            else:
                df.index = df.index.tz_convert('UTC')
            df = df[['Open', 'High', 'Low', 'Close', 'Volume']]
            if isinstance(df.columns, pd.MultiIndex):
                df.columns = df.columns.droplevel(1)
            df.columns = [f"{ticker}_{col}" for col in df.columns]
            dfs[ticker] = df
        except Exception as e:
            print(f"  -> ERROR fetching {ticker}: {e}")
    if not dfs:
        raise ValueError("Failed to fetch data for all given tickers.")
    print(f"\nMerging datasets...")
    combined = pd.concat(dfs.values(), axis=1).sort_index()
    combined = combined.ffill(limit=72)
    print(f"Fetches {len(combined)} total hourly observations.")
    return combined

def preprocess_features(df, target_prefix='TRUMP35336-USD'):
    print(f"\nPreprocessing RL state features for target: {target_prefix}...")
    df = df.copy()
    close_col = f'{target_prefix}_Close'
    vol_col = f'{target_prefix}_Volume'
    if close_col not in df.columns:
        raise ValueError(f"Target close column '{close_col}' not found in dataframe.")
    original_len_before_drop = len(df)
    df = df.dropna(subset=[close_col])
    print(f"  -> Dropped {original_len_before_drop - len(df)} rows where
```



```

{close_col} was missing.")
    # 1. Price Momentum Features (Log Returns)
    df['feature_log_return_1h'] = np.log(df[close_col] /
df[close_col].shift(1))
    df['feature_log_return_4h'] = np.log(df[close_col] /
df[close_col].shift(4))
    df['feature_log_return_24h'] = np.log(df[close_col] /
df[close_col].shift(24))
    # 2. Volatility Features
    df['feature_volatility_24h'] =
df['feature_log_return_1h'].rolling(window=24).std()
    # 3. Volume Features
    if vol_col in df.columns:
        df['feature_log_volume'] = np.log1p(df[vol_col])
    # 4. Intraday Timing (Cyclical Encoding)
    hours = df.index.hour
    df['feature_hour_sin'] = np.sin(2 * np.pi * hours / 24.0)
    df['feature_hour_cos'] = np.cos(2 * np.pi * hours / 24.0)
    days = df.index.dayofweek
    df['feature_day_sin'] = np.sin(2 * np.pi * days / 7.0)
    df['feature_day_cos'] = np.cos(2 * np.pi * days / 7.0)
    # 5. Clean up NaNs
    original_len = len(df)
    features_to_check = [col for col in df.columns if
col.startswith('feature_')]
    df = df.dropna(subset=features_to_check)
    print(f"Dropped {original_len - len(df)} warm-up rows.")
    print(f"Final preprocessed shape: {df.shape}")
    return df

if __name__ == "__main__":
    target_ticker = 'TRUMP35336-USD'
    tickers = [target_ticker, 'BTC-USD', 'ETH-USD', 'GC=F', 'SI=F', 'ES=F']
    DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)),
'data')
    RAW_DIR = os.path.join(DATA_DIR, 'raw')
    PROCESSED_DIR = os.path.join(DATA_DIR, 'processed')
    os.makedirs(RAW_DIR, exist_ok=True)
    os.makedirs(PROCESSED_DIR, exist_ok=True)
    try:
        raw_df = fetch_data(tickers, interval='1h', period='730d')
        raw_df.to_csv(os.path.join(RAW_DIR, 'raw_hourly_pull.csv'))
        processed_df = preprocess_features(raw_df,
target_prefix=target_ticker)
        processed_df.to_csv(os.path.join(PROCESSED_DIR,
'rl_dataset_hourly.csv'))
        btc_df = preprocess_features(raw_df, target_prefix='BTC-USD')
        btc_df.to_csv(os.path.join(PROCESSED_DIR,
'rl_dataset_btc_hourly.csv'))
    except Exception as e:
        print(f"\nCRITICAL ERROR in pipeline: {e}")

```

A.2 Trading Environment (env.py)

```

import gymnasium as gym
from gymnasium import spaces
import numpy as np
import pandas as pd
import os

class TradingEnv(gym.Env):
    metadata = {"render_modes": ["human"]}

    def __init__(self, data_path_or_df=None, risk_aversion=1.0,
tx_cost=0.001, initial_capital=10000.0,
                normalize=False, norm_stats=None):
        super(TradingEnv, self).__init__()
        if isinstance(data_path_or_df, pd.DataFrame):
            self.df = data_path_or_df.copy()
        elif isinstance(data_path_or_df, str):
            self.df = pd.read_csv(data_path_or_df, index_col=0,
parse_dates=True)
        else:
            default_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
                'data', 'processed',
'rl_dataset_hourly.csv')
            self.df = pd.read_csv(default_path, index_col=0,
parse_dates=True)
            self.risk_aversion = risk_aversion
            self.tx_cost = tx_cost
            self.initial_capital = initial_capital
            self.normalize = normalize
            # Discrete Action Space: 5 Actions
            # 0: Heavy Sell (0%), 1: Light Sell (-25%), 2: Hold, 3: Light Buy
(+25%), 4: Heavy Buy (100%)
            self.action_space = spaces.Discrete(5)
            self.feature_cols = [
                'feature_log_return_1h', 'feature_log_return_4h',
'feature_log_return_24h',
                'feature_volatility_24h', 'feature_log_volume',
                'feature_hour_sin', 'feature_hour_cos', 'feature_day_sin',
'feature_day_cos',
            ]
            missing_cols = [c for c in self.feature_cols if c not in
self.df.columns]
            if missing_cols:
                raise ValueError(f"Missing required features in dataframe:
{missing_cols}")
            self.n_features = len(self.feature_cols) + 1 # +1 for position
            self.observation_space = spaces.Box(low=-np.inf, high=np.inf,
shape=(self.n_features,),
dtype=np.float32)
            self.current_step = 0
            self.position = 0.0
            self.portfolio_value = self.initial_capital
            self.returns_1h = self.df['feature_log_return_1h'].values
            self.features_matrix = self.df[self.feature_cols].values
            if self.normalize:
                if norm_stats is not None:

```

```

        self.norm_stats = norm_stats
    else:
        self.norm_stats = {
            'mean': self.features_matrix.mean(axis=0),
            'std': self.features_matrix.std(axis=0),
        }
        self.norm_stats['std'][self.norm_stats['std'] < 1e-8] = 1.0
    else:
        self.norm_stats = None

def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    self.current_step = 0
    self.position = 0.0
    self.portfolio_value = self.initial_capital
    return self._get_obs(), {}

def _action_to_position(self, action):
    if action == 0: return 0.0
    elif action == 1: return self.position * 0.75
    elif action == 2: return self.position
    elif action == 3: return self.position + (1.0 - self.position) *
0.25
    elif action == 4: return 1.0
    else: raise ValueError(f"Invalid action: {action}")

def step(self, action):
    if self.current_step >= len(self.df) - 1:
        return self._get_obs(), 0.0, True, False, {}
    target_position = self._action_to_position(action)
    trade_size = abs(target_position - self.position)
    cost = trade_size * self.tx_cost * self.portfolio_value
    self.position = target_position
    self.current_step += 1
    price_return = self.returns_1h[self.current_step]
    portfolio_return = self.position * price_return
    self.portfolio_value *= (1 + portfolio_return)
    self.portfolio_value -= cost
    # Quadratic Utility:  $r_t = R_t - \lambda * R_t^2 - \text{cost}$ 
    cost_pct = cost / max(self.portfolio_value, 1e-8)
    reward = portfolio_return - (self.risk_aversion * (portfolio_return
** 2)) - cost_pct
    terminated = self.current_step >= len(self.df) - 1
    if self.portfolio_value <= 0:
        self.portfolio_value = 0
        reward -= 1.0
        terminated = True
    info = {'portfolio_value': self.portfolio_value, 'position':
self.position,
            'action': action, 'reward': reward, 'cost': cost}
    return self._get_obs(), float(reward), terminated, False, info

def _get_obs(self):
    row_features = self.features_matrix[self.current_step]
    if self.normalize and self.norm_stats is not None:
        row_features = (row_features - self.norm_stats['mean']) /
self.norm_stats['std']

```

```
obs = np.append(row_features, self.position)
return obs.astype(np.float32)
```

A.3 Tabular Q-Learning Agent (td_agent.py)

```
import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
from env import TradingEnv

class QLearningAgent:
    def __init__(self, action_space_size=5, alpha=0.1, gamma=0.99,
                  epsilon_start=1.0, epsilon_min=0.01, epsilon_decay=0.995):
        self.action_space_size = action_space_size
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon_start
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        # State Discretization: 3 * 3 * 3 * 2 = 54 states
        self.state_dims = (3, 3, 3, 2)
        self.q_table = np.zeros(self.state_dims + (self.action_space_size,))

    def _discretize_state(self, obs, env):
        return_1h = obs[0]
        volatility_24h = obs[3]
        hour_sin = obs[5]
        position = obs[9]
        if return_1h < -0.5:      return_bin = 0
        elif return_1h > 0.5:    return_bin = 2
        else:                   return_bin = 1
        if volatility_24h < -0.5: vol_bin = 0
        elif volatility_24h > 0.5: vol_bin = 2
        else:                   vol_bin = 1
        if position < 0.33:      pos_bin = 0
        elif position > 0.66:    pos_bin = 2
        else:                   pos_bin = 1
        time_bin = 0 if hour_sin < 0 else 1
        return (return_bin, vol_bin, pos_bin, time_bin)

    def choose_action(self, obs, env, evaluation=False):
        state = self._discretize_state(obs, env)
        if not evaluation and np.random.rand() < self.epsilon:
            return env.action_space.sample()
        else:
            return np.argmax(self.q_table[state])

    def learn(self, obs, action, reward, next_obs, env, done):
        state = self._discretize_state(obs, env)
        next_state = self._discretize_state(next_obs, env)
        best_next_action = np.argmax(self.q_table[next_state])
```

```

        td_target = reward + self.gamma * self.q_table[next_state]
[best_next_action] * (not done)
        td_error = td_target - self.q_table[state][action]
        self.q_table[state][action] += self.alpha * td_error

ACTION_NAMES = ["Heavy Sell", "Light Sell", "Hold", "Light Buy", "Heavy
Buy"]

def split_dataset(csv_path, train_ratio=0.7):
    df = pd.read_csv(csv_path, index_col=0, parse_dates=True)
    split_idx = int(len(df) * train_ratio)
    return df.iloc[:split_idx], df.iloc[split_idx:]

def run_experiment(config, progress_cb=None):
    data_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
'data', 'processed')
    dataset = config.get('dataset', 'trump')
    csv_name = 'rl_dataset_btc_hourly.csv' if dataset == 'btc' else
'rl_dataset_hourly.csv'
    data_path = os.path.join(data_dir, csv_name)
    train_ratio = config.get('train_ratio', 0.7)
    train_df, test_df = split_dataset(data_path, train_ratio)
    risk_aversion = config.get('risk_aversion', 1.0)
    tx_cost = config.get('tx_cost', 0.001)
    train_env = TradingEnv(data_path_or_df=train_df,
risk_aversion=risk_aversion,
                        tx_cost=tx_cost, normalize=True)
    test_env = TradingEnv(data_path_or_df=test_df,
risk_aversion=risk_aversion,
                        tx_cost=tx_cost, normalize=True,
norm_stats=train_env.norm_stats)
    agent = QLearningAgent(
        alpha=config.get('alpha', 0.1), gamma=config.get('gamma', 0.99),
        epsilon_start=config.get('epsilon_start', 1.0),
        epsilon_min=config.get('epsilon_min', 0.01),
        epsilon_decay=config.get('epsilon_decay', 0.995),
    )
    episodes = config.get('episodes', 20)
    rewards_hist, portfolios_hist = [], []
    for ep in range(episodes):
        obs, _ = train_env.reset()
        done, total_reward = False, 0.0
        while not done:
            action = agent.choose_action(obs, train_env)
            next_obs, reward, terminated, truncated, _ =
train_env.step(action)
            done = terminated or truncated
            agent.learn(obs, action, reward, next_obs, train_env, done)
            obs = next_obs
            total_reward += reward
        if agent.epsilon > agent.epsilon_min:
            agent.epsilon *= agent.epsilon_decay
        rewards_hist.append(total_reward)
        portfolios_hist.append(train_env.portfolio_value)
        if progress_cb:
            progress_cb(ep + 1, episodes, {'reward': total_reward,
'portfolio': train_env.portfolio_value, 'epsilon':

```

```

agent.epsilon})
# Evaluation
obs, _ = test_env.reset()
done, total_reward = False, 0.0
action_counts = {i: 0 for i in range(5)}
eval_portfolios = []
while not done:
    action = agent.choose_action(obs, test_env, evaluation=True)
    action_counts[action] += 1
    obs, reward, terminated, truncated, info = test_env.step(action)
    done = terminated or truncated
    total_reward += reward
    eval_portfolios.append(test_env.portfolio_value)
action_dist = {ACTION_NAMES[a]: c for a, c in action_counts.items()}
return {
    'hyperparameters': {'alpha': agent.alpha, 'gamma': agent.gamma,
        'epsilon_start': config.get('epsilon_start', 1.0),
        'epsilon_decay': agent.epsilon_decay, 'episodes': episodes},
    'env_config': {'risk_aversion': risk_aversion, 'tx_cost': tx_cost,
        'train_ratio': train_ratio, 'dataset': dataset},
    'training': {'rewards': rewards_hist, 'portfolios':
portfolios_hist},
    'evaluation': {'final_portfolio': test_env.portfolio_value,
        'total_reward': total_reward, 'action_distribution':
action_dist,
        'portfolio_curve': eval_portfolios},
    'q_table': agent.q_table,
}

```

A.4 Double DQN Agent (`dqn_agent.py`)

```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import collections
import random
import os
import pandas as pd
from env import TradingEnv

DEFAULTS = {
    'lr': 1e-4, 'gamma': 0.99, 'batch_size': 64, 'replay_size': 50000,
    'target_update': 500, 'hidden_dim': 64, 'dropout_rate': 0.1,
    'weight_decay': 1e-5, 'state_noise': 0.01, 'episodes': 50,
    'epsilon_start': 1.0, 'epsilon_end': 0.01, 'epsilon_decay': 0.95,
    'risk_aversion': 0.5, 'tx_cost': 0.001, 'train_ratio': 0.7,
}

class QNetwork(nn.Module):
    def __init__(self, obs_dim, action_dim, hidden_dim=64,
dropout_rate=0.1):

```

```

    super().__init__()
    self.fc1 = nn.Linear(obs_dim, hidden_dim)
    self.dropout = nn.Dropout(p=dropout_rate) if dropout_rate > 0 else
nn.Identity()
    self.fc2 = nn.Linear(hidden_dim, hidden_dim)
    self.fc3 = nn.Linear(hidden_dim, action_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout(x)
        return self.fc3(x)

class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = collections.deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        state, action, reward, next_state, done = map(np.array, zip(*batch))
        return state, action, reward, next_state, done

    def __len__(self):
        return len(self.buffer)

class DQNAgent:
    def __init__(self, obs_dim, action_dim, cfg):
        self.action_dim = action_dim
        self.epsilon = cfg.get('epsilon_start', 1.0)
        self.epsilon_end = cfg.get('epsilon_end', 0.01)
        self.epsilon_decay = cfg.get('epsilon_decay', 0.95)
        self.gamma = cfg.get('gamma', 0.99)
        self.batch_size = cfg.get('batch_size', 64)
        self.target_update = cfg.get('target_update', 500)
        self.state_noise = cfg.get('state_noise', 0.01)
        self.device = torch.device("mps" if
torch.backends.mps.is_available()
        else ("cuda" if torch.cuda.is_available() else "cpu"))
        hidden = cfg.get('hidden_dim', 64)
        drop = cfg.get('dropout_rate', 0.1)
        self.q_net = QNetwork(obs_dim, action_dim, hidden,
drop).to(self.device)
        self.target_net = QNetwork(obs_dim, action_dim, hidden,
drop).to(self.device)
        self.target_net.load_state_dict(self.q_net.state_dict())
        self.target_net.eval()
        self.optimizer = optim.Adam(self.q_net.parameters(),
lr=cfg.get('lr', 1e-4), weight_decay=cfg.get('weight_decay', 1e-
5))

        self.memory = ReplayBuffer(cfg.get('replay_size', 50000))
        self.steps_done = 0

    def choose_action(self, state, evaluation=False):

```



```

        if not evaluation and random.random() < self.epsilon:
            return random.randint(0, self.action_dim - 1)
        state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        self.q_net.eval()
        with torch.no_grad():
            q = self.q_net(state_t)
        self.q_net.train()
        return q.argmax().item()

def learn(self):
    if len(self.memory) < self.batch_size:
        return
    states, actions, rewards, next_states, dones =
self.memory.sample(self.batch_size)
    states = torch.FloatTensor(states).to(self.device)
    actions = torch.LongTensor(actions).unsqueeze(1).to(self.device)
    rewards = torch.FloatTensor(rewards).unsqueeze(1).to(self.device)
    next_states = torch.FloatTensor(next_states).to(self.device)
    dones = torch.FloatTensor(dones).unsqueeze(1).to(self.device)
    q_values = self.q_net(states).gather(1, actions)
    with torch.no_grad():
        next_actions = self.q_net(next_states).argmax(1).unsqueeze(1)
        next_q = self.target_net(next_states).gather(1, next_actions)
        target = rewards + self.gamma * next_q * (1 - dones)
    loss = nn.MSELoss()(q_values, target)
    self.optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_value_(self.q_net.parameters(), 1.0)
    self.optimizer.step()
    self.steps_done += 1
    if self.steps_done % self.target_update == 0:
        self.target_net.load_state_dict(self.q_net.state_dict())

ACTION_NAMES = ["Heavy Sell", "Light Sell", "Hold", "Light Buy", "Heavy
Buy"]

def split_dataset(csv_path, train_ratio=0.7):
    df = pd.read_csv(csv_path, index_col=0, parse_dates=True)
    split_idx = int(len(df) * train_ratio)
    return df.iloc[:split_idx], df.iloc[split_idx:]

def run_experiment(config, progress_cb=None):
    cfg = {**DEFAULTS, **config}
    data_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
'data', 'processed')
    dataset = cfg.get('dataset', 'trump')
    csv_name = 'rl_dataset_btc_hourly.csv' if dataset == 'btc' else
'rl_dataset_hourly.csv'
    data_path = os.path.join(data_dir, csv_name)
    train_df, test_df = split_dataset(data_path, cfg['train_ratio'])
    train_env = TradingEnv(data_path_or_df=train_df,
        risk_aversion=cfg['risk_aversion'], tx_cost=cfg['tx_cost'],
normalize=True)
    test_env = TradingEnv(data_path_or_df=test_df,
        risk_aversion=cfg['risk_aversion'], tx_cost=cfg['tx_cost'],
        normalize=True, norm_stats=train_env.norm_stats)
    obs_dim, action_dim = 10, 5

```



```

agent = DQNAgent(obs_dim, action_dim, cfg)
episodes = cfg['episodes']
noise = cfg['state_noise']
rewards_hist, portfolios_hist = [], []
for ep in range(episodes):
    obs, _ = train_env.reset()
    rp = np.random.uniform(0.0, 1.0)
    train_env.position = rp
    obs[-1] = rp
    done, total_reward = False, 0.0
    while not done:
        noisy = obs + np.random.normal(0, noise, obs.shape) if noise > 0
    else obs
        action = agent.choose_action(noisy)
        next_obs, reward, terminated, truncated, _ =
train_env.step(action)
        done = terminated or truncated
        noisy_next = next_obs + np.random.normal(0, noise,
next_obs.shape) if noise > 0 else next_obs
        agent.memory.push(noisy, action, reward, noisy_next, done)
        agent.learn()
        obs = next_obs
        total_reward += reward
        if agent.epsilon > agent.epsilon_end:
            agent.epsilon = max(agent.epsilon_end, agent.epsilon *
agent.epsilon_decay)
        rewards_hist.append(total_reward)
        portfolios_hist.append(train_env.portfolio_value)
        if progress_cb:
            progress_cb(ep + 1, episodes, {'reward': total_reward,
            'portfolio': train_env.portfolio_value, 'epsilon':
agent.epsilon})
    # Evaluation
    obs, _ = test_env.reset()
    done, total_reward = False, 0.0
    action_counts = {i: 0 for i in range(5)}
    eval_portfolios = []
    while not done:
        action = agent.choose_action(obs, evaluation=True)
        action_counts[action] += 1
        obs, reward, terminated, truncated, _ = test_env.step(action)
        done = terminated or truncated
        total_reward += reward
        eval_portfolios.append(test_env.portfolio_value)
    action_dist = {ACTION_NAMES[a]: c for a, c in action_counts.items()}
    return {
        'hyperparameters': {k: cfg[k] for k in [
            'lr', 'gamma', 'batch_size', 'replay_size', 'target_update',
            'hidden_dim', 'dropout_rate', 'weight_decay', 'state_noise',
            'epsilon_start', 'epsilon_end', 'epsilon_decay', 'episodes']},
        'env_config': {k: cfg[k] for k in ['risk_aversion', 'tx_cost',
'train_ratio', 'dataset']},
        'training': {'rewards': rewards_hist, 'portfolios':
portfolios_hist},
        'evaluation': {'final_portfolio': test_env.portfolio_value,
            'total_reward': total_reward, 'action_distribution':
action_dist,

```

```

        'portfolio_curve': eval_portfolios},
        'model_state': agent.q_net.state_dict(),
    }

```

A.5 Baseline Strategies (**baselines.py**)

```

import numpy as np
import pandas as pd
import os
from env import TradingEnv

ACTION_NAMES = ["Heavy Sell", "Light Sell", "Hold", "Light Buy", "Heavy Buy"]

def split_dataset(csv_path, train_ratio=0.7):
    df = pd.read_csv(csv_path, index_col=0, parse_dates=True)
    split_idx = int(len(df) * train_ratio)
    return df.iloc[:split_idx], df.iloc[split_idx:]

def _resolve_data_path(config):
    data_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                             'data', 'processed')
    dataset = config.get('dataset', 'trump')
    csv_name = 'rl_dataset_btc_hourly.csv' if dataset == 'btc' else
    'rl_dataset_hourly.csv'
    return os.path.join(data_dir, csv_name)

def _evaluate(env, action_fn):
    obs, _ = env.reset()
    done, total_reward, step = False, 0.0, 0
    action_counts = {i: 0 for i in range(5)}
    portfolios = []
    while not done:
        action = action_fn(obs, step, env)
        action_counts[action] += 1
        obs, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        total_reward += reward
        portfolios.append(env.portfolio_value)
        step += 1
    return {
        'final_portfolio': env.portfolio_value,
        'total_reward': total_reward,
        'action_distribution': {ACTION_NAMES[a]: c for a, c in
    action_counts.items()},
        'portfolio_curve': portfolios,
    }

def _buy_and_hold(config):
    data_path = _resolve_data_path(config)
    _, test_df = split_dataset(data_path, config.get('train_ratio', 0.7))
    env = TradingEnv(data_path_or_df=test_df,

```

```

        risk_aversion=config.get('risk_aversion', 1.0),
        tx_cost=config.get('tx_cost', 0.001), normalize=True)
def action_fn(obs, step, env):
    return 4 if step == 0 else 2
return _evaluate(env, action_fn)

def _sma_crossover(config):
    data_path = _resolve_data_path(config)
    _, test_df = split_dataset(data_path, config.get('train_ratio', 0.7))
    env = TradingEnv(data_path_or_df=test_df,
                     risk_aversion=config.get('risk_aversion', 1.0),
                     tx_cost=config.get('tx_cost', 0.001), normalize=True)
    short_window = config.get('sma_short', 12)
    long_window = config.get('sma_long', 48)
    returns = test_df['feature_log_return_1h'].values
    sma_short = pd.Series(returns).rolling(short_window,
min_periods=1).mean().values
    sma_long = pd.Series(returns).rolling(long_window,
min_periods=1).mean().values
    def action_fn(obs, step, env):
        if step < long_window: return 2
        if sma_short[step] > sma_long[step]: return 4
        elif sma_short[step] < sma_long[step]: return 0
        return 2
    return _evaluate(env, action_fn)

def _random_agent(config):
    data_path = _resolve_data_path(config)
    _, test_df = split_dataset(data_path, config.get('train_ratio', 0.7))
    env = TradingEnv(data_path_or_df=test_df,
                     risk_aversion=config.get('risk_aversion', 1.0),
                     tx_cost=config.get('tx_cost', 0.001), normalize=True)
    def action_fn(obs, step, env):
        return env.action_space.sample()
    return _evaluate(env, action_fn)

STRATEGIES = {'buy_hold': _buy_and_hold, 'sma_crossover': _sma_crossover,
'random': _random_agent}

def run_experiment(config, progress_cb=None):
    strategy = config.get('strategy', 'buy_hold')
    fn = STRATEGIES[strategy]
    evaluation = fn(config)
    hp = {'strategy': strategy}
    if strategy == 'sma_crossover':
        hp['sma_short'] = config.get('sma_short', 12)
        hp['sma_long'] = config.get('sma_long', 48)
    return {
        'hyperparameters': hp,
        'env_config': {'risk_aversion': config.get('risk_aversion', 1.0),
            'tx_cost': config.get('tx_cost', 0.001),
            'train_ratio': config.get('train_ratio', 0.7),
            'dataset': config.get('dataset', 'trump')},
        'training': {'rewards': [], 'portfolios': []},
        'evaluation': evaluation,
    }

```

A.6 Streamlit Dashboard (dashboard.py)

```
import streamlit as st
import json, os, glob, datetime
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd

st.set_page_config(page_title="TRUMP Coin RL Dashboard", layout="wide")
RUNS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "data",
                        "runs")
os.makedirs(RUNS_DIR, exist_ok=True)
ACTION_NAMES = ["Heavy Sell", "Light Sell", "Hold", "Light Buy", "Heavy Buy"]

def save_run(model_name, result):
    ts = datetime.datetime.now().strftime("%Y-%m-%d_%H%M%S")
    ds = result.get('env_config', {}).get('dataset', 'trump')
    run_id = f"{model_name}_{ds}_{ts}"
    if 'q_table' in result:
        np.save(os.path.join(RUNS_DIR, f"{run_id}.npy"),
                result.pop('q_table'))
    if 'model_state' in result:
        import torch
        torch.save(result.pop('model_state'), os.path.join(RUNS_DIR, f"{run_id}.pt"))
    payload = {"run_id": run_id, "model": model_name, "timestamp": ts,
               **result}
    path = os.path.join(RUNS_DIR, f"{run_id}.json")
    with open(path, "w") as f:
        json.dump(payload, f, indent=2, default=str)
    return run_id

def load_all_runs():
    runs = []
    for fp in sorted(glob.glob(os.path.join(RUNS_DIR, "*.json"))):
        with open(fp) as f:
            runs.append(json.load(f))
    return runs

page = st.sidebar.radio("Navigation", [
    "🚀 Run Experiment", "🇺🇸 Explore Past Runs", "🔍 Inspect Q-Values"])

# — PAGE 1: RUN EXPERIMENT —
if page == "🚀 Run Experiment":
    st.title("🚀 Run Experiment")
    ALL_MODELS = ["Tabular Q-Learning", "Double DQN", "Buy & Hold", "SMA Crossover", "Random Agent"]
    model = st.radio("Model", ALL_MODELS, horizontal=True)
    is_baseline = model in ("Buy & Hold", "SMA Crossover", "Random Agent")
    st.markdown("----")
    col_env, col_model = st.columns(2)
    with col_env:
```

```

st.subheader("Environment")
dataset = st.radio("Dataset", ["TRUMP", "BTC"], horizontal=True)
risk_aversion = st.slider("Risk Aversion ( $\lambda$ )", 0.0, 5.0, 0.5, 0.1)
tx_cost = st.number_input("Transaction Cost", 0.0000, 0.0100,
0.0010, 0.0001, format="%.4f")
train_ratio = st.slider("Train / Test Split", 0.50, 0.90, 0.70,
0.05)
with col_model:
    if model == "Tabular Q-Learning":
        st.subheader("Q-Learning Hyperparameters")
        alpha = st.slider("Learning Rate ( $\alpha$ )", 0.01, 1.0, 0.10, 0.01)
        gamma = st.slider("Discount Factor ( $\gamma$ )", 0.80, 1.0, 0.99, 0.01)
        eps_decay = st.slider("Epsilon Decay", 0.50, 0.999, 0.80, 0.001)
        episodes = st.number_input("Episodes", 5, 200, 20)
    elif model == "Double DQN":
        st.subheader("DQN Hyperparameters")
        lr = st.select_slider("Learning Rate", [1e-5, 3e-5, 1e-4, 3e-4,
1e-3], value=1e-4)
        gamma = st.slider("Discount Factor ( $\gamma$ )", 0.80, 1.0, 0.99, 0.01)
        hidden_dim = st.select_slider("Hidden Dim", [32, 64, 128, 256],
value=64)
        dropout_rate = st.slider("Dropout Rate", 0.0, 0.5, 0.1, 0.05)
        weight_decay = st.select_slider("Weight Decay", [0.0, 1e-6, 1e-
5, 1e-4, 1e-3], value=1e-5)
        state_noise = st.slider("State Noise ( $\sigma$ )", 0.0, 0.10, 0.01,
0.005)
        eps_decay = st.slider("Epsilon Decay", 0.80, 0.999, 0.95, 0.001)
        episodes = st.number_input("Episodes", 5, 200, 50)
    elif model == "SMA Crossover":
        st.subheader("SMA Parameters")
        sma_short = st.number_input("Short Window (hours)", 4, 100, 12)
        sma_long = st.number_input("Long Window (hours)", 12, 500, 48)
    elif model == "Buy & Hold":
        st.subheader("Buy & Hold")
        st.info("Buys 100% on step 0, then holds.")
    elif model == "Random Agent":
        st.subheader("Random Agent")
        st.info("Takes a random action every step.")
    btn_label = "▶ Run Evaluation" if is_baseline else "▶ Start
Training"
    st.markdown("----")
    if st.button(btn_label, type="primary", use_container_width=True):
        progress_bar = st.progress(0)
        status_area = st.empty()
        def on_progress(ep, total, m):
            progress_bar.progress(ep / total)
            status_area.text(f"Episode {ep}/{total} - Reward:
{m['reward']:.2f} "
                            f"| Portfolio: ${m['portfolio']:.2f} |  $\epsilon$ :
{m['epsilon']:.3f}")
        with st.spinner("Running..."):
            if model == "Tabular Q-Learning":
                from td_agent import run_experiment
                config = dict(alpha=alpha, gamma=gamma,
epsilon_decay=eps_decay,
                                episodes=int(episodes),
risk_aversion=risk_aversion,

```

```

        tx_cost=tx_cost, train_ratio=train_ratio,
dataset=dataset.lower())
    result = run_experiment(config, progress_cb=on_progress)
    elif model == "Double DQN":
        from dqn_agent import run_experiment
        config = dict(lr=lr, gamma=gamma, hidden_dim=hidden_dim,
            dropout_rate=dropout_rate,
weight_decay=weight_decay,
            state_noise=state_noise,
epsilon_decay=eps_decay,
            episodes=int(episodes),
risk_aversion=risk_aversion,
            tx_cost=tx_cost, train_ratio=train_ratio,
dataset=dataset.lower())
    result = run_experiment(config, progress_cb=on_progress)
    else:
        from baselines import run_experiment
        strategy_map = {"Buy & Hold": "buy_hold",
            "SMA Crossover": "sma_crossover", "Random
Agent": "random"}
        config = dict(strategy=strategy_map[model],
risk_aversion=risk_aversion,
            tx_cost=tx_cost, train_ratio=train_ratio,
dataset=dataset.lower())
        if model == "SMA Crossover":
            config['sma_short'] = int(sma_short)
            config['sma_long'] = int(sma_long)
            result = run_experiment(config)
        progress_bar.progress(1.0)
        status_area.success("✅ Complete!")
        model_key_map = {"Tabular Q-Learning": "q_learning", "Double DQN":
"dqn",
            "Buy & Hold": "buy_hold", "SMA Crossover":
"sma_crossover",
            "Random Agent": "random"}
        run_id = save_run(model_key_map[model], result)
        st.info(f"Run saved as `{run_id}`")
        ev = result['evaluation']
        c1, c2, c3 = st.columns(3)
        c1.metric("Test Portfolio", f"${ev['final_portfolio']:.2f}",
            f"{{{(ev['final_portfolio'] / 10000) - 1} * 100:+.1f}%")
        c2.metric("Test Reward", f"{ev['total_reward']:.4f}")
        total_actions = sum(ev['action_distribution'].values())
        dominant = max(ev['action_distribution'],
key=ev['action_distribution'].get)
        c3.metric("Dominant Action", dominant,
            f"{ev['action_distribution'][dominant] /
max(total_actions, 1) * 100:.0f}%")

# — PAGE 2: EXPLORE PAST RUNS —
elif page == "📊 Explore Past Runs":
    st.title("📊 Explore Past Runs")
    runs = load_all_runs()
    if not runs:
        st.info("No runs saved yet.")
        st.stop()
    table_data = []

```

```

for r in runs:
    ev = r.get('evaluation', {})
    table_data.append({
        'Run ID': r['run_id'], 'Model': r['model'],
        'Dataset': r.get('env_config', {}).get('dataset',
'trump').upper(),
        'Test Portfolio': f"${ev.get('final_portfolio', 0):.2f}",
        'Test Reward': f"{ev.get('total_reward', 0):.4f}",
    })
    st.dataframe(pd.DataFrame(table_data), use_container_width=True,
hide_index=True)
    run_ids = [r['run_id'] for r in runs]
    selected = st.multiselect("Select runs to compare", run_ids,
                             default=run_ids[-min(2, len(run_ids)):])

    if not selected:
        st.stop()
    selected_runs = [r for r in runs if r['run_id'] in selected]
    st.subheader("Training Reward Curves")
    fig = go.Figure()
    for r in selected_runs:
        rewards = r.get('training', {}).get('rewards', [])
        fig.add_trace(go.Scatter(x=list(range(1, len(rewards) + 1)),
y=rewards,
                                mode='lines', name=r['run_id'])))
    fig.update_layout(xaxis_title="Episode", yaxis_title="Reward",
height=400)
    st.plotly_chart(fig, use_container_width=True)

# — PAGE 3: INSPECT Q-VALUES —
elif page == "🔍 Inspect Q-Values":
    st.title("🔍 Inspect Q-Values")
    runs = load_all_runs()
    if not runs:
        st.info("No runs saved yet.")
        st.stop()
    ql_runs = [r for r in runs if r['model'] == 'q_learning'
               and os.path.exists(os.path.join(RUNS_DIR, f"
{r['run_id']}.npz"))]
    dqn_runs = [r for r in runs if r['model'] == 'dqn'
               and os.path.exists(os.path.join(RUNS_DIR, f"
{r['run_id']}.pt"))]
    if not ql_runs and not dqn_runs:
        st.warning("No runs with saved model artifacts found.")
        st.stop()
    available = []
    if ql_runs: available.append("Tabular Q-Learning")
    if dqn_runs: available.append("Double DQN")
    model_type = st.radio("Model Type", available, horizontal=True)
    if model_type == "Tabular Q-Learning":
        run_id = st.selectbox("Select Run", [r['run_id'] for r in ql_runs])
        q_table = np.load(os.path.join(RUNS_DIR, f"{run_id}.npz"))
        momentum_labels = {0: "Negative", 1: "Flat", 2: "Positive"}
        volatility_labels = {0: "Low", 1: "Medium", 2: "High"}
        position_labels = {0: "Mostly Cash", 1: "Balanced", 2: "Mostly
TRUMP"}
        time_labels = {0: "Night", 1: "Day"}
        col1, col2 = st.columns(2)

```



```

        with col1:
            momentum = st.selectbox("Momentum", [0, 1, 2],
                                    format_func=lambda x:
momentum_labels[x])
            volatility = st.selectbox("Volatility", [0, 1, 2],
                                    format_func=lambda x:
volatility_labels[x])
            position = st.selectbox("Position", [0, 1, 2],
                                    format_func=lambda x:
position_labels[x])
            time_bin = st.selectbox("Time of Day", [0, 1],
                                    format_func=lambda x: time_labels[x])
            q_vals = q_table[momentum, volatility, position, time_bin]
            greedy = int(np.argmax(q_vals))
            with col2:
                st.subheader("Q-Values for Selected State")
                colors = ['#ff6b6b' if i != greedy else '#51cf66' for i in
range(5)]
                fig = go.Figure(go.Bar(x=ACTION_NAMES, y=q_vals,
marker_color=colors,
                                text=[f"{v:.4f}" for v in q_vals],
textposition='outside'))
                fig.update_layout(height=350, yaxis_title="Q-Value")
                st.plotly_chart(fig, use_container_width=True)
            elif model_type == "Double DQN":
                import torch
                from dqn_agent import QNetwork
                run_id = st.selectbox("Select Run", [r['run_id'] for r in dqn_runs])
                run_meta = next(r for r in dqn_runs if r['run_id'] == run_id)
                hp = run_meta.get('hyperparameters', {})
                net = QNetwork(obs_dim=10, action_dim=5,
                                hidden_dim=int(hp.get('hidden_dim', 64)),
                                dropout_rate=float(hp.get('dropout_rate', 0.1)))
                net.load_state_dict(torch.load(os.path.join(RUNS_DIR, f"
{run_id}.pt"),
                                map_location='cpu',
weights_only=True))
                net.eval()
                col1, col2 = st.columns(2)
                with col1:
                    ret_1h = st.slider("1h Log Return", -0.10, 0.10, 0.0, 0.001)
                    ret_4h = st.slider("4h Log Return", -0.20, 0.20, 0.0, 0.005)
                    ret_24h = st.slider("24h Log Return", -0.50, 0.50, 0.0, 0.01)
                    vol_24h = st.slider("Volatility (24h)", 0.0, 0.10, 0.02, 0.005)
                    log_vol = st.slider("Log Volume", 10.0, 30.0, 20.0, 0.5)
                    hour_sin = st.slider("Hour Sin", -1.0, 1.0, 0.0, 0.1)
                    hour_cos = st.slider("Hour Cos", -1.0, 1.0, 1.0, 0.1)
                    day_sin = st.slider("Day Sin", -1.0, 1.0, 0.0, 0.1)
                    day_cos = st.slider("Day Cos", -1.0, 1.0, 1.0, 0.1)
                    pos = st.slider("Portfolio Position", 0.0, 1.0, 0.0, 0.05)
                state = torch.FloatTensor([ret_1h, ret_4h, ret_24h, vol_24h,
log_vol,
                                hour_sin, hour_cos, day_sin, day_cos,
pos]).unsqueeze(0)
                with torch.no_grad():
                    q_vals = net(state).squeeze().numpy()
                    greedy = int(np.argmax(q_vals))

```



```
with col2:
    st.subheader("Q-Values")
    colors = ['#ff6b6b' if i != greedy else '#51cf66' for i in
range(5)]
    fig = go.Figure(go.Bar(x=ACTION_NAMES, y=q_vals,
marker_color=colors,
                        text=[f"{v:.4f}" for v in q_vals],
textposition='outside'))
    fig.update_layout(height=350, yaxis_title="Q-Value")
    st.plotly_chart(fig, use_container_width=True)
```